CASSA OBSERVATORY • HANDS-ON SESSION

Photometric Pipeline Participant Handbook

Everything you need for the session: install the pipeline on your own machine, put a night of observatory data where it belongs, and work through five notebooks that take raw frames to a calibrated magnitude — with an error bar you can defend.

YOU WILL PRODUCE	NOTEBOOKS	DISK NEEDED	PYTHON
one star's mag ± error	5, run in order	~1 GB	3.10 or newer

Contents

1 Before you start

Do the install **before** the session, not during it. Building the environment downloads a few hundred megabytes and takes ten to twenty minutes on a normal connection; doing it in the room costs everyone the first hour.

WHAT	WHY
A laptop running Linux, macOS or Windows	One script per platform; all three install natively. WSL also works on Windows if you already use it.
∼1 GB free disk	155 MB for the night's raw frames (they come with the repository), ∼265 MB for what the pipeline writes, plus the environment.
Internet, at least once	Installing, and Phase 3's reference-catalog query. Both cache, so a second pass runs offline.
Comfort with a terminal	You will type about six commands. No prior astronomy software needed.

> **BRING THIS**
>
> Your assigned star — the instructor gives you either a catalog `NUMBER` or an RA/Dec pair. Measuring it is the session's deliverable.

2 Install

One route, on every operating system: get **Miniforge**, then run one script. Windows gets there through WSL — see below, and do that part first.

> **WHY MINIFORGE AND NOT PLAIN PIP**
>
> Phase 2 plate-solves your images against a star catalog, and every solver that can do it is a compiled binary rather than a Python package. Miniforge is what supplies one, along with prebuilt versions of the whole scientific stack, so nothing has to compile on your laptop.

Which platform you are on

YOUR MACHINE	ROUTE	NOTES
Linux, x86-64	`./install.sh`	The reference platform.
Linux, ARM (aarch64)	`./install.sh --conda`	Works, but only with ASTAP — nothing else is published for ARM Linux.
macOS, Intel	`./install.sh`	Identical to Linux.
macOS, Apple Silicon	`./install.sh`	Identical. See the Gatekeeper note below.
Windows	`.\install.ps1`	Works natively now. WSL also fine if you already use it.

Windows

Windows installs natively. Get Miniforge (or use a Miniconda you already have), then from PowerShell in the cloned repository:

```
.\install.ps1
cassa-doctor
```

It does what `install.sh` does on Linux and macOS: builds the environment, installs the pipeline, fetches ASTAP — the one plate solver published for Windows — registers the Jupyter kernel, and checks its own work.

> **WSL STILL WORKS, IF YOU PREFER IT**
>
> WSL is real x86-64 Linux, so every instruction in this handbook applies unchanged inside it. `.\install.ps1 -Wsl` prints the setup steps. If you go that way, keep the repository and the work directory in your WSL home — your Windows drives appear under `/mnt/c`, and reducing a night across `/mnt` is several times slower.

Step 1 — Install Miniforge

Miniforge is a minimal conda that defaults to the conda-forge channel. If you already have conda or mamba, skip this step.

The installer name is built from your own machine, so this is the same pair of commands on Linux, on macOS (Intel and Apple Silicon alike), and inside WSL:

```
curl -L -O "https://github.com/conda-forge/miniforge/releases/latest/\
download/Miniforge3-$(uname)-$(uname -m).sh"
bash Miniforge3-$(uname)-$(uname -m).sh
```

Accept the defaults, then **close and reopen your terminal** so `conda` is on your PATH. Check it:

```
conda --version
```

On macOS you also need Apple's command-line tools, once, for `git`:

```
xcode-select --install
```

Step 2 — Clone and install

Identical everywhere:

```
git clone https://github.com/cassaiub/observatory.git
cd observatory/photometric_pipeline
./install.sh
```

The script builds a conda environment called `cassa-photometry` from `environment.yml`, installs the pipeline into it, adds whichever plate solver your platform can run, and finishes by checking its own work. Expect ten to twenty minutes and a few hundred megabytes.

On ARM Linux add `--conda`: the in-process solver publishes no ARM wheel, so the conda route plus ASTAP is the combination that works there.

Step 3 — Activate and verify

```
conda activate cassa-photometry
cassa-doctor
```

`cassa-doctor` is the one command that proves the install: it reports your platform, the package version, the environment, which plate solver you have and which one a run would use, and where the sky data will come from. **Run it before the session, and bring the output if anything is not green.** You need to run `conda activate cassa-photometry` in every new terminal.

CHECK THE KERNEL NAME BEFORE YOU START

The installer registers a Jupyter kernel called **Python (CASSA photometry)**. When a notebook opens, check that name appears in the top-right corner; if it does not, use *Kernel → Change Kernel*.
This is the single most common way the session goes wrong, and it looks like a broken install rather than a wrong kernel: the notebook reports `ModuleNotFoundError: No module named 'cassa_photometry'` on a machine where the install clearly succeeded. Installing packages sets up an *environment*; it does not by itself make Jupyter use it.

WHICH SOLVER YOU GET, AND WHY IT DOES NOT MATTER

The installer tries three, in order, and the first that installs wins: **ASTAP** (a sub-megabyte binary that works on every platform, including ARM), then Astrometry.net's `solve-field`, then an in-process Python solver. `cassa-doctor` tells you which one you have.
Your results do not depend on the answer. The one thing ASTAP does not report — the astrometric residual `ASTRMS`, which Phase 3 uses to size its catalog cross-match — the pipeline measures for itself, so every backend produces it. If your neighbour has a different solver, your catalogs are still comparable.

THERE IS NOTHING TO DOWNLOAD IN ADVANCE

Plate solving matches your stars against a sky catalog, and those catalogs are large — 859 MB for ASTAP, some 34 GB for Astrometry.net. You need neither. The pipeline reads your frame's pointing and field size, works out which pieces that field needs, and fetches only those: about **6 MB** for ASTAP or 165 MB for Astrometry.net, cached under `~/.cache/cassa-photometry/`. Pointing somewhere new later costs only the pieces the new field adds.

MACOS: GATEKEEPER MAY QUARANTINE ASTAP

If `cassa-doctor` finds the ASTAP binary but a solve fails with a security warning, macOS has quarantined the downloaded file. Clear it once: `xattr -d com.apple.quarantine "$(command -v astap_cli)"`

3 Where the data goes

The dataset ships with the repository. `workshop/raw/` already holds one real night — 77 frames, 155 MB — so there is nothing to fetch and nothing to generate:

```
workshop/raw/
`-- 20230822                    <- the night, as the telescope named it
    |-- Bias/                     10 frames,  0 s
    |-- Dark/                     10 frames, 60 s
    |-- Flat/                     43 frames,  3 s
    `-- Light/NGC7331/            14 frames, 60 s
        `-- 2023_08_23T04.02.30.985Z_B.fits
```

NGC 7331, observed on 2023-08-23 with an iTelescope **CDK700** and an **Andor DU934P** CCD, 1024×1024. It is real data, with the mess that implies — which is the point of using it.

> TWO THINGS ABOUT THIS NIGHT YOU WILL MEET, AND SHOULD
>
> **The header's plate scale is wrong.** `SECPIX` claims 0.4 arcsec/px; the truth is 0.5905 arcsec/px, a 32% error. A wrong scale hint makes a solve *fail* where a missing one only makes it slow, so the pipeline solves anyway and then tells you the header lied. You will see that warning in notebook 03.
> **There is no gain or read noise in the header.** Neither `EGAIN` nor `READNOIS` is present, so phase 1 assumes 1.0 e-/ADU and 10 e- and says so for every frame. Every uncertainty downstream is therefore an estimate rather than a measurement. That warning is the pipeline being honest, not a fault to fix.

If you bring *your own* night instead, copy the whole dated folder into `workshop/raw/` unchanged. `RAW_DIR` is searched **recursively**, so the tree is read exactly as the acquisition software delivered it — and a single flat directory of frames works too, if that is what you are given.

> FOLDERS DO NOT DECIDE ANYTHING
>
> Every frame is classified by its `IMAGETYP`, `FILTER` and `EXPTIME` headers, never by the folder it sits in. A frame filed under the wrong folder is still reduced as whatever it actually is — notebook 01 tells you when the two disagree.

Filters in this night

B V R

Johnson-Cousins B, V and R at 60 s — five science frames in B and R, four in V. Flats exist for all three (18 in B, 15 in R, 10 in V). There is no luminance filter in this night.

What the pipeline writes

Everything lands under `workshop/work/`, one directory per phase. Re-running a phase replaces its products in place rather than piling up copies.

DIRECTORY	WRITTEN BY	CONTENTS
`work/phase1`	notebook 02	`calibrated_*.fits` — SCI/ERR/DQ, in electrons
`work/phase2`	notebook 03	`Master_*.fits` — stacked and WCS-solved
`work/phase3`	notebook 04	`*_catalog.csv`, `*_fluxcal.fits`, segmentation maps

If your data lives somewhere else

Point at it instead of copying — the notebooks honour these environment variables:

```
export RAW_DIR=/path/to/the/night
export WORK_DIR=/path/for/outputs
```

4 Running the notebooks

Activate the environment, then start Jupyter from the notebooks directory:

```
conda activate cassa-photometry
cd observatory/photometric_pipeline/workshop/notebooks
jupyter lab
```

Run them in order. Notebook 00 checks your environment and 01 reads only the raw frames, but 02, 03 and 04 each consume the previous notebook's output. Within a notebook, run every cell from the top — later cells depend on names defined earlier.

The first cell of every notebook

Each notebook opens with the same config cell. It resolves `RAW_DIR`, `WORK_DIR` and the per-phase directories, then prints what it found — including your raw tree, directory by directory. If those paths look wrong, stop and fix them there; everything downstream inherits them.

Filling in the blanks

The exercises are partly written for you. Lines you must complete are marked:

```
gain = FILL_IN        # TODO 1: e-/ADU as Phase 1 resolved it
```

The surrounding code is scaffolding — leave it alone. Each blank takes one expression. If you run a cell without filling them you get `NameError: name 'FILL_IN' is not defined`, which names the exact line still waiting for you. Nothing fails silently.

Reading the answer tables

Every exercise heading carries a small table naming the one or two values it wants, with the expected answer beside each. Compare your numbers against it — a mismatch is worth raising in the room, because it usually means something real about your data rather than a mistake in your arithmetic.

5 Notebook by notebook

00 Setup ~5 min

Four checks that your machine can do the session at all. Run every cell.

What you are checking

- The package imports and reports a version.
- **A plate solver is available** — notebook 00 lists all three and names the one a run would use. Two showing as unavailable is normal: you need one, and which one does not change your results.
- Where the sky data will come from. An empty cache is fine — the tiles a field needs are fetched during the solve.
- The raw tree is found, and the frame counts per directory match the night that ships (10 bias, 10 dark, 43 flat, 14 light) or the night you copied in.

STOP IF the raw tree shows zero frames, or `cassa-doctor` reports no usable plate solver. Neither is recoverable later.

01 Raw frame health ~10 min

Are these frames worth reducing? The pipeline will happily reduce bad calibration frames and produce plausible numbers that are quietly meaningless — this is where that gets caught.

What it does

- Inventories every frame by type, filter and exposure, and flags any frame whose folder disagrees with its header.
- Measures each frame once — level, scatter, saturated fraction, temperature — and plots every frame's signal level against the ADC ceiling.
- Runs sixteen checks and prints a verdict.

The verdict

PROCEED Everything passed. Go on to 02.

CAUTION Nothing blocking, but the result will be weaker than it looks. Note the warnings — they explain later numbers.

STOP A blocking problem. Each one names a way the reduction will be *wrong* rather than merely noisy, and none can be repaired afterwards. Raise it with an instructor.

RECORD the verdict, and the headline of any FAIL.

02 Phase 1 — Calibration ~20 min

Bias, dark and flat come off; the frame is converted to electrons; and the error budget is seeded. Output is `calibrated_*.fits` carrying SCI, ERR and DQ.

The run itself takes several minutes for a full night — start it, then read the exercise text while it works.

Exercises

`Ex 1 · 3 blanks` **What did calibration actually change?** Find the instrumental pedestal that was removed, and the real sky left behind. Both in electrons.

`Ex 2 · 2 blanks` **Is the ERR plane just Poisson + read noise?** Find the ratio of actual to predicted error, and what the excess is.

RECORD four numbers: pedestal, sky, ratio, extra term.

03 Phase 2 — Integration & WCS ~20 min

Frames are aligned, stacked with inverse-variance weighting, and plate-solved against real stars. Output is `Master_*.fits` with a WCS.

Exercises

`Ex 1 · 2 blanks` **How much deeper is the stack?** Find the depth gain and the number of frames stacked. Compare against $\sqrt{N}$.

`Ex 2 · 2 blanks` **What did the plate solve buy us?** Find the plate scale in arcsec/pixel and the field of view in arcmin.

WATCH FOR a depth gain that falls short of $\sqrt{N}$. That is real, not an error: warping resamples pixels, which correlates neighbours, and correlated noise does not average down as fast as the formula assumes.

04 Phase 3 — Photometry & zero point ~25 min

Sources are detected and measured, a zero point is derived by cross-matching against a reference catalog, and a catalog with magnitudes and errors is written. This is the notebook that needs internet.

Exercises

Ex 1 · 2 blanks **Check the zero-point arithmetic.** Recompute magnitude and error from flux and the header; find the largest residual, and which term dominates for the brightest star.

Ex 2 · 2 blanks **Your assigned star.** Fill in `ASSIGNED_NUMBER` or `ASSIGNED_RADEC` at the top of the cell, then report the magnitude, its uncertainty, and which half of the error budget dominates.

IF THERE IS NO ZERO POINT

Phase 3 writes `_fluxcal.fits` only when it measures one. Without it the catalog still exists, carrying instrumental magnitudes, and the notebook falls back to those automatically — every shape you are asked about survives, only the zero of the scale is arbitrary. Exercise 1 skips itself and says so.

6 What to hand in

One line, from Exercise 2 of notebook 04:

ITEM	EXAMPLE
Which star	`NUMBER 118, RA 339.19496, Dec 34.46298`
Magnitude ± uncertainty	`12.162 ± 0.004`
Dominant error term	`zero point`
Filter and frames stacked	`V, 4 frames`

QUOTE IT PROPERLY

A magnitude without an uncertainty is not a measurement, and one written to more digits than its uncertainty supports is a claim you cannot defend. Round the value to the precision the error allows — the notebook does this for you, and it is worth understanding why.

7 When something breaks

WHAT YOU SEE	WHAT IT MEANS & WHAT TO DO
`NameError: name 'FILL_IN' is not defined`	A blank you have not filled yet. The traceback names the line.
`NameError` on a name from the config cell	Your kernel imported the config before it was last edited. **Kernel** → **Restart**, then run from the first cell.
Notebook 01 says **STOP**	A calibration frame is unusable. Do not reduce past it — the numbers will look fine and be wrong. Raise it in the room.
`solver: solve-field not on PATH`	Not a fault. Three backends can solve, and `cassa-doctor`'s `solver: in use` line says which one you have. Only a problem if `solver: any` *fails* — that means none is installed.
`no star database covering this field`	ASTAP could not fetch the sky tiles for your pointing. Check your connection; the tiles are about 6 MB and are fetched during the solve.
macOS: ASTAP found, but the solve fails with a security warning	Gatekeeper has quarantined the downloaded binary. Clear it once with `xattr -d com.apple.quarantine "$(command -v astap_cli)"`
Phase 3: *zero point failed*, no `_fluxcal.fits`	The cross-match found no usable stars in your image. The catalog is still written with instrumental magnitudes and the exercises adapt. Worth investigating, not worth blocking on.
Phase 3 hangs or times out	It is querying an online reference catalog. Check your connection; the query caches, so a second run is fast.
`conda not found`	Miniforge is not installed, or you did not reopen the terminal after installing it. On Windows, make sure you are inside the Ubuntu (WSL) window, not PowerShell.
A phase takes far longer than the estimate	Normal on a laptop for a full night. Watch the progress bars; they move per frame. On WSL, also check the repository is in your Linux home rather than under `/mnt/c` — that alone can cost several times the runtime.

Still stuck? `cassa-doctor` prints the state of the whole install in one screen. Bring that output rather than a description.